

0.76.9. Un déclencheur en (0, 0) : le B tourne en carré — en simple

Le 0.76.2 en une ligne : `tourne_carre(0, 10, 8, ALPHABET[1], 4, 250)`. Plus de `bpas`, `bx`, `bimages` : la console s'en occupe.

1. Ce qu'il faut comprendre

C'est le 0.76.2, en une ligne. Tout le bloc des mouvements (le chronomètre `bimages`, le compteur de tour `bpas`, les quatre `un_pas`, `bx`, `by`) est remplacé par **une fonction de la console**.

Ce qui est nouveau ici : `tourne_carre(numero, x, y, tuile, cote, vitesse)`. À appeler **à chaque image**. Au premier appel, la lettre apparaît en (`x`, `y`), un coin du carré ; ensuite, toutes les `vitesse` millisecondes, elle fait **un pas** : `cote` pas à droite, puis en bas, puis à gauche, puis en haut, et elle recommence, sans fin.

- `numero` (0 à 3) : la console retient, pour chaque numéro, **où en est** la lettre (sa place, son pas dans le tour, son attente). C'est ce qui remplace `bx`, `by`, `bpas` et `bimages`.
- `cote` : 4, un côté de 4 pas, comme au 0.76.2. • `vitesse` : 250 ms par pas, écrite en clair.

Elle ne bloque pas : comme `un_pas`, elle fait au plus un pas et rend la main. Le A suit toujours la croix.

Elle remplace aussi le `poser` du 0.76.1 : c'est son premier appel qui fait apparaître le B.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **2536 octets** de cartouche.

1. UNE VARIABLE — X

```
uint8_t x = 10;    // le A part de (10, 0)
```

2. UNE VARIABLE — Y

```
uint8_t y = 0;
```

3. UNE VARIABLE — ACTIF

```
uint8_t actif = 0;    // 0 : pas encore ; 1 : le A est passé par (0, 0)
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

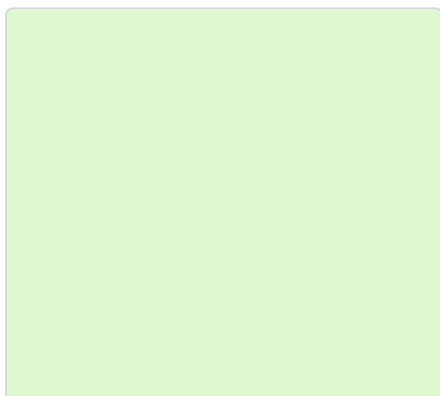
```
int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 250); // le A suit la croix
        nombre(0, 17, x);                      // sa position
        nombre(4, 17, y);

        // Le déclencheur, en simple : pas besoin de « && actif == 0 ».
        // Remettre actif à 1 quand il vaut déjà 1 ne change rien.
        if (x == 0 && y == 0) {
            actif = 1;
        }

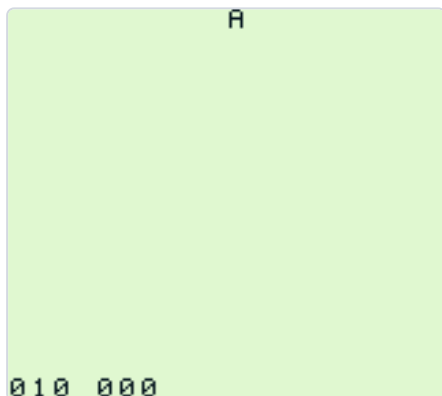
        // Tout ce qui se passe après le déclencheur, une ligne par lettre :
        if (actif == 1) {
            tourne_carre(0, 10, 8, ALPHABET[1], 4, 250); // le B tourne en carré
        }
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Comme au 0.76.2 : en (0, 0), le B apparaît au milieu et tourne en carré.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le B tourne : (14, 8), (14, 12), (10, 12)

Ensuite : **0.76.10. Un déclencheur en (0, 0) : le carré plus rapide — en simple** — niveau 0 sur 10. · [gameboy3](#) — écrire une cartouche Game Boy en C++